

AI ASSISTED CODING ASSIGNMENT – 10.4

2403A53002

SATHWIK VEMULAWADA

24BTCAICYB01-2-1

Task 1: Syntax and Error Detection.

Task: Identify and fix syntax, indentation, and variable errors in the given script.

CODE 1:



The screenshot shows a Jupyter Notebook interface. On the left, there is a cell icon with a red exclamation mark and the text "[1] 0s". The main area displays a Python code snippet with syntax errors highlighted by red wavy lines under the function name and the variable 'result'. The code is as follows:

```
def add numbers(a,b)
return=a+b
return result
print(add_numbers(10,20))
```

Below the code, a syntax error message is displayed with a red arrow pointing to the start of the function definition:

```
File "/tmp/ipython-input-2019714464.py", line 1
def add numbers(a,b)
    ^
SyntaxError: expected '('
```

At the bottom, there is a dashed line and a section labeled "Next steps:" with a button that says "Explain error".

PROMPT:

V Identify and fix syntax, indentation, and variable errors in the given script.
Expected Output: • Corrected code with proper syntax (: after function, fixed variable name, corrected function call).

CODE 2 AND OUTPUT:

```
/ command palette (Ctrl+Shift+P)

[2]
✓ 0s ▶ def add_numbers(a, b):
      result = a + b
      return result

      print(add_numbers(10, 20))

⇨ 30
```

EXPLANATION:

The code defines a simple Python function and then calls it to demonstrate its usage.

* `def add_numbers(a, b):`: This line defines a function named `add_numbers` that takes two arguments, `a` and `b`. The colon indicates the start of the function's code block. `result = a + b`: Inside the function, this line adds the values of `a` and `b` and stores the sum in a variable called `result`. `return result`: This line returns the value stored in the `result` variable from the function. `print(add_numbers(10, 20))`: This line calls the `add_numbers` function with the arguments `10` and `20`. The function returns `30`, and then the `print()` function displays this result to the console.

The code defines a simple Python function and then calls it to demonstrate its usage.

ERROR CORRECTIONS:

**`def add_numbers(a, b):`: This line defines a function named `add_numbers` that takes two arguments, `a` and `b`. The colon indicates the start of the function's code block.
`result = a + b`: Inside the function, this line adds the values of `a` and `b` and

stores the sum in a variable called result.
return result: This line returns the value stored in the result variable from the function.
print(add_numbers(10, 20)): This line calls the add_numbers function with the arguments 10 and 20. The function returns 30, and then the print() function displays this result to the console..

Task 2: Logical and Performance Issue Review.

Task: Optimize inefficient logic while keeping the result correct.

CODE 1:

```
[4]
❗ Os
▶ def find_duplicates(nums):
    duplicates = []
    for i in range(len(nums)):
    for j in range(len(nums)):
    if i != j and nums[i] == nums[j] and nums[i] not in duplicates:
    duplicates.append(nums[i])
    return duplicates
    numbers = [1,2,3,2,4,5,1,6,1,2]
    print(find_duplicates(numbers))

📄 File "/tmp/ipython-input-638266264.py", line 4
    for j in range(len(nums)):
    ^
IndentationError: expected an indented block after 'for' statement on line 3
```

Next steps: [Explain error](#)

PROMPT:

```
Gemini X
V Optimize inefficient logic while keeping the result correct.Expected Output:
More efficient duplicate detection (e.g., using sets)
```

CODE 2 AND OUTPUT:

```
[5]
✓ 0s
def find_duplicates(nums):
    seen = set()
    duplicates = set()
    for num in nums:
        if num in seen:
            duplicates.add(num)
        else:
            seen.add(num)
    return list(duplicates)

numbers = [1,2,3,2,4,5,1,6,1,2]
print(find_duplicates(numbers))

[1, 2]
```

EXPLANATION:

This code defines a function called `find_duplicates` that efficiently finds duplicate numbers in a list using sets. Here's a breakdown:

- `def find_duplicates(nums):`: This defines the function `find_duplicates` that takes one argument, a list of numbers called `nums`.
- `seen = set()`: This initializes an empty set called `seen`. This set will store the numbers encountered so far.
- `duplicates = set()`: This initializes another empty set called `duplicates`. This set will store the numbers that are found to be duplicates.
- `for num in nums:`: This starts a loop that iterates through each number in the input list `nums`.
- `if num in seen:`: Inside the loop, this checks if the current number `num` is already present in the `seen` set.
- `duplicates.add(num)`: If `num` is in `seen`, it means it's a duplicate, so it's added to the `duplicates` set.
- `else: seen.add(num)`: If `num` is not in `seen`, it's the first time we've encountered this number, so it's added to the `seen` set.
- `return list(duplicates)`: After the loop finishes, the function converts the `duplicates` set into a list and returns it.
- `numbers = [1,2,3,2,4,5,1,6,1,2]`: This line creates a list of numbers to test the function.
- `print(find_duplicates(numbers))`: This line calls the `find_duplicates` function with the `numbers` list and prints the returned list of duplicates.

Using sets for `seen` and `duplicates`

makes this approach efficient because checking for membership in a set (num in seen) is very fast.

ERROR CORRECTIONS:

 Please explain this error:

 File `"/tmp/ipython-input-638266264.py"`, line 4 `for j in ran...`

 The error in the selected cell is an `IndentationError`. This means that the lines of code inside the `for` loops are not indented correctly. Python uses indentation to define blocks of code. I will fix the indentation and update the cell.

Task 3: Code Refactoring for Readability.

Task: Refactor messy code into clean, PEP 8-compliant, well-structured code.

CODE 1:

[6]
 0s

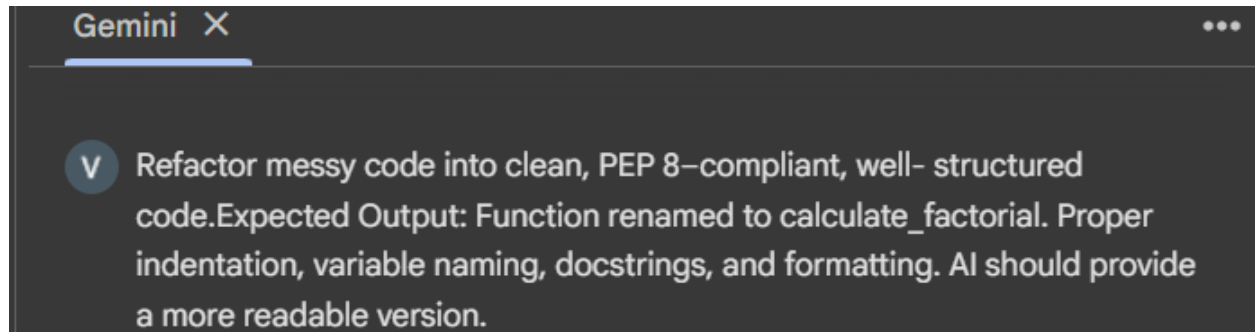


```
def c(n):  
    x=1  
    for i in range(1,n+1):  
        x=x*i  
    return x  
print(c(5))
```

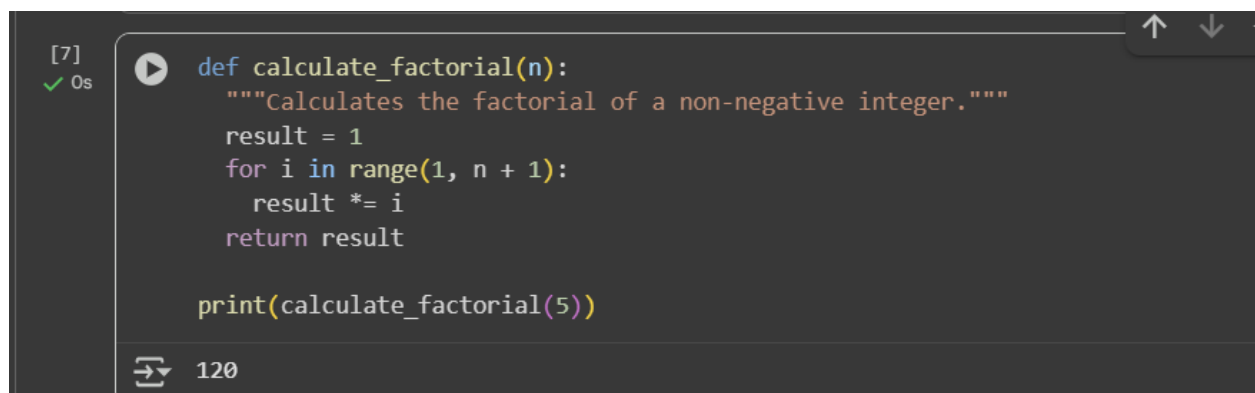
 File `"/tmp/ipython-input-2642679034.py"`, line 2
 x=1
 ^
`IndentationError`: expected an indented block after function definition on line 1

Next steps: [Explain error](#)

PROMPT:



CODE 2:

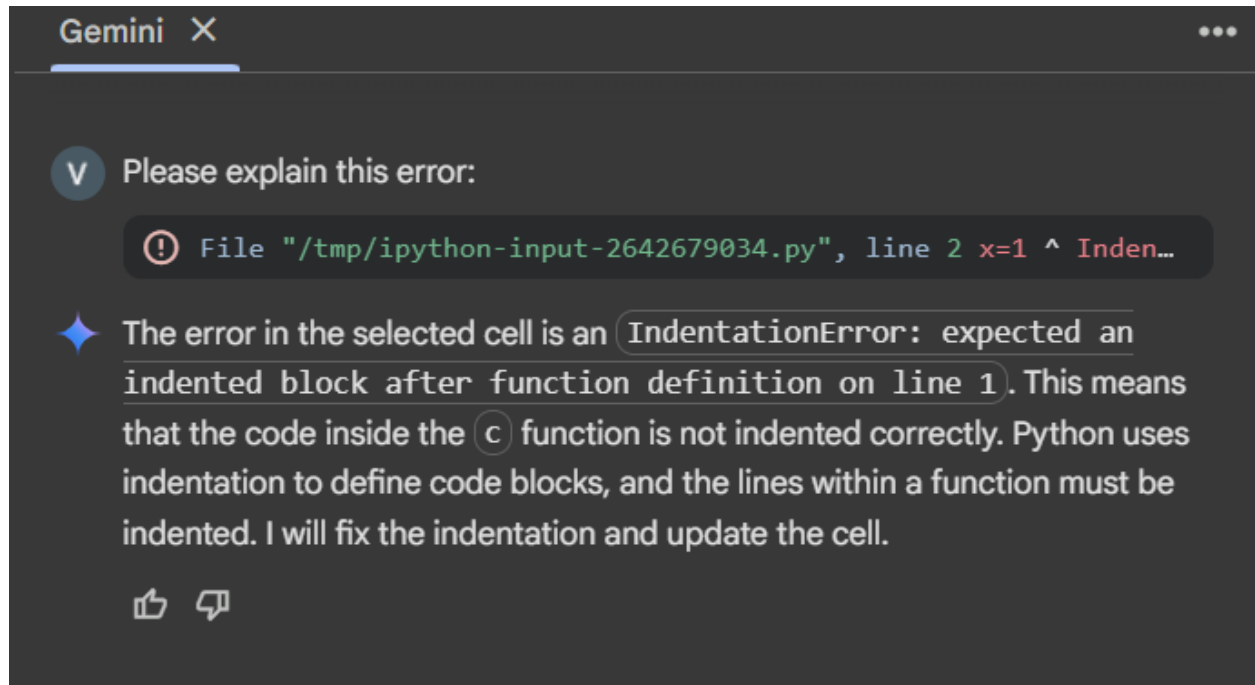


EXPLANATION:

The selected code defines a function to calculate the factorial of a non-negative integer and then demonstrates its use. Here's a breakdown:

- `def calculate_factorial(n):`: This line defines a function named `calculate_factorial` that takes one argument, `n`.
- `"""Calculates the factorial of a non-negative integer."""`: This is a docstring, which explains what the function does. It's a good practice to include docstrings for better code documentation.
- `result = 1`: This line initializes a variable `result` to 1. This is the starting value for the factorial calculation. The factorial of 0 is 1, and for any positive integer, we'll multiply this result by subsequent numbers.
- `for i in range(1, n + 1):`: This starts a for loop that iterates through a sequence of numbers starting from 1 up to and including `n`.
- `result *= i`: Inside the loop, this line multiplies the current value of `result` by the loop variable `i` and updates `result` with the new value. This is the core of the factorial calculation.
- `return result`: After the loop finishes, the function returns the final calculated result, which is the factorial of `n`.
- `print(calculate_factorial(5))`: This line calls the `calculate_factorial` function with the argument 5 and prints the returned value (which is the factorial of 5, i.e., 120). The refactored code is more readable due to the descriptive function and variable names, as well as the added docstring.

ERROR CORRECTIONS:



Task 4: Security and Error Handling Enhancement.

Task: Add security practices and exception handling to the code.

CODE 1:

```
[8] 0s
import sqlite3
def get_user_data(user_id):
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    query = f"SELECT * FROM users WHERE id = {user_id};" #
    Potential SQL injection risk
    cursor.execute(query)
    result = cursor.fetchall()
    conn.close()
    return result
user_input = input("Enter user ID: ")
print(get_user_data(user_input))
```

File "/tmp/ipython-input-1610427666.py", line 3
conn = sqlite3.connect("users.db")
^
IndentationError: expected an indented block after function definition on line 2

Next steps: [Explain error](#)

PROMPT:

Gemini X

V Add security practices and exception handling to the code.Expected Output: Safe query using parameterized SQL (? placeholders). Try-except block for database errors. Input validation before query execution.

CODE'S 2 AND OUTPUT:

[15]
✓ 16s

```
import sqlite3
def get_user_data(user_id):
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    query = "SELECT * FROM users WHERE id = ?;"
    try:
        cursor.execute(query, (user_id,))
        result = cursor.fetchall()
    except sqlite3.Error as e:
        print(f"Database error: {e}")
        result = []
    finally:
        conn.close()
    return result
user_input = input("Enter user ID: ")
try:
    user_id_int = int(user_input)
    print(get_user_data(user_id_int))
except ValueError:
    print("Invalid input. Please enter an integer for the user ID.")
```

↵ Enter user ID: 145
Database error: no such table: users

```

import sqlite3
def get_user_data(user_id):
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    # Use parameterized query with '?' placeholder
    query = "SELECT * FROM users WHERE id = ?;"
    try:
        # Pass the user_id as a tuple to execute
        cursor.execute(query, (user_id,))
        result = cursor.fetchall()
    except sqlite3.Error as e:
        print(f"Database error: {e}")
        result = []
    finally:
        conn.close()
    return result

user_input = input("Enter user ID: ")
try:
    user_id_int = int(user_input)
    print(get_user_data(user_id_int))
except ValueError:
    print("Invalid input. Please enter an integer for the user ID.")

```

```

Enter user ID: 1
Database error: no such table: users
[]

```

```
import sqlite3

def get_user_data(user_id):
    conn = None # Initialize conn to None
    result = [] # Initialize result to empty list
    try:
        conn = sqlite3.connect("users.db")
        cursor = conn.cursor()
        query = "SELECT * FROM users WHERE id = ?;"
        cursor.execute(query, (user_id,))
        result = cursor.fetchall()
    except sqlite3.Error as e:
        print(f"Database error: {e}")
        result = [] # Ensure result is empty on error
    finally:
        if conn: # Check if conn was successfully created
            conn.close()
    return result

user_input = input("Enter user ID: ")
try:
    user_id_int = int(user_input)
    print(get_user_data(user_id_int))
except ValueError:
    print("Invalid input. Please enter an integer for the user ID.")
```

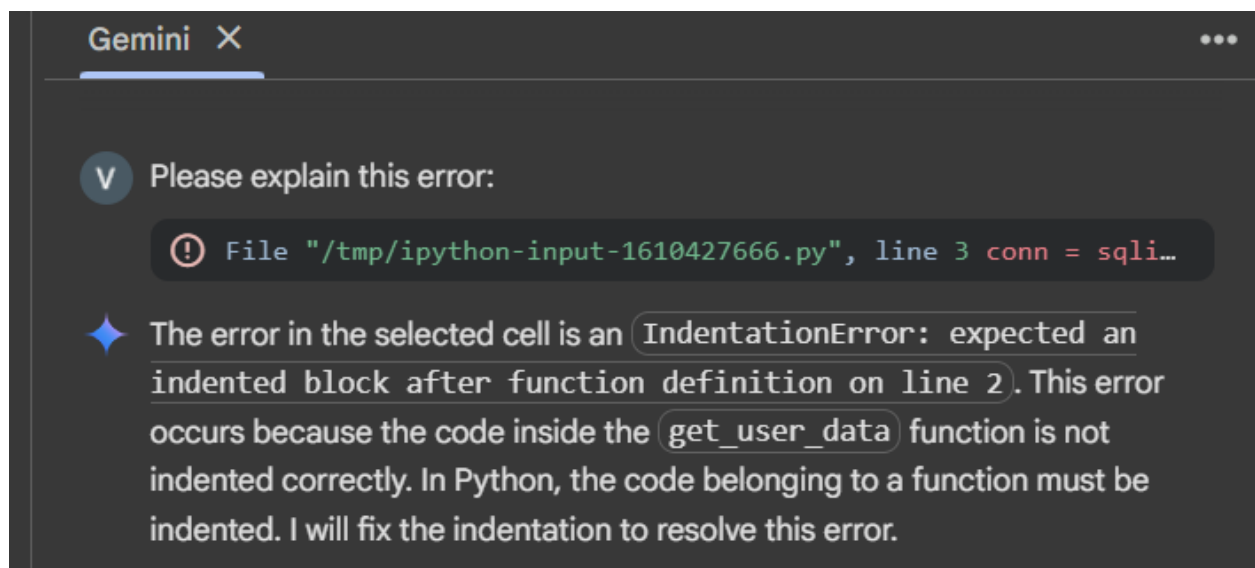
Enter user ID: 154
Database error: no such table: users
[]

EXPLANATION:

This code is designed to safely retrieve user data from a SQLite database while preventing SQL injection and handling potential errors. Here's a breakdown of the code in cell import sqlite3: This line imports the sqlite3 module, which provides an interface for working with SQLite databases in Python. def get_user_data(user_id):: This defines the function get_user_data that takes one argument, user_id. conn = None: Initializes the conn variable to None. This variable will hold the database connection object. result = []: Initializes the result variable to an empty list. This variable will store the data fetched from the database. try:: This block starts a try...except...finally structure to handle potential errors. The code inside the try block is executed, and if an error occurs, the corresponding except block is executed. The finally block is always executed. conn =

`sqlite3.connect("users.db")`: This line establishes a connection to the SQLite database file named "users.db". `cursor = conn.cursor()`: This creates a cursor object, which is used to execute SQL commands. `query = "SELECT * FROM users WHERE id = ?;"`: This defines the SQL query. Notice the `?` placeholder for the `user_id`. This is crucial for preventing SQL injection. `cursor.execute(query, (user_id,))`: This line executes the SQL query. The `user_id` is passed as a tuple `(user_id,)` as the second argument to `execute`. SQLite replaces the `?` placeholder with the value of `user_id`, ensuring that the input is treated as data, not executable code. `result = cursor.fetchall()`: This fetches all the rows returned by the query and stores them in the `result` variable as a list of tuples. `except sqlite3.Error as e`: This block catches any `sqlite3.Error` that might occur during the database operations. `print(f"Database error: {e}")`: If a database error occurs, this line prints an informative error message. `result = []`: This ensures that the result is an empty list if a database error occurs, preventing unexpected behavior. `finally`: This block is executed regardless of whether an exception occurred or not. `if conn`: This checks if the `conn` variable is not `None`, meaning a database connection was successfully established. `conn.close()`: If a connection exists, this line closes the database connection, releasing the resources. `return result`: The function returns the result (either the fetched data or an empty list in case of an error). `user_input = input("Enter user ID: ")`: This line prompts the user to enter a user ID. `try`: This starts another `try...except` block to handle potential errors during input conversion. `user_id_int = int(user_input)`: This attempts to convert the user's input (which is initially a string) into an integer. `print(get_user_data(user_id_int))`: If the conversion is successful, this line calls the `get_user_data` function with the integer user ID and prints the returned result. `except ValueError`: This block catches a `ValueError` that occurs if the user's input cannot be converted to an integer. `print("Invalid input. Please enter an integer for the user ID.")`: If a `ValueError` occurs, this line prints an error message to the user. In summary, this code is a significantly improved version of the original, incorporating essential security and error handling practices for safer database interactions.

ERROR CORRECTIONS:



Task 5: Automated Code Review Report Generation.

Task: Generate a review report for this messy code.

CODE 1:

```
[17] def calc(x,y,z):
      if z=="add":
      return x+y
      elif z=="sub": return x-y
      elif z=="mul":
      return x*y
      elif z=="div":
      return x/y
      else: print("wrong")
      print(calc(10,5,"add"))
      print(calc(10,0,"div"))
```

File "/tmp/ipython-input-4161972190.py", line 2
if z=="add":
^
IndentationError: expected an indented block after function definition on line 1

Next steps: [Explain error](#)

PROMPT:

Gemini X

V Generate a review report for this messy code.Expected Output: AI-generated review report should mention: o Missing docstrings o Inconsistent formatting (indentation, inline return) o Missing error handling for division by zero o Non-descriptive function/variable names o Suggestions for readability and PEP 8 compliance

CODE REVIEW:

Issues Identified:

- **Missing Docstrings:** The function `calc` lacks a docstring, which makes it unclear what the function does without reading the code.
- **Inconsistent Formatting:**
 - Indentation is incorrect, leading to an `IndentationError`.
 - The `elif z=="sub": return x-y` line uses an inline return, which is inconsistent with the block returns for other operations and reduces readability.

- **Missing Error Handling:** There is no specific error handling for division by zero when `z` is "div" and `y` is 0. This will cause a `ZeroDivisionError`.
- **Non-Descriptive Names:**
 - The function name `calc` is not descriptive; a name like `perform_operation` or `basic_calculator` would be more informative.
 - Variable names `x`, `y`, and `z` are not descriptive; names like `num1`, `num2`, and `operation` would improve readability.
- **PEP 8 Compliance:** The code does not fully comply with PEP 8 guidelines regarding indentation, spacing, and naming conventions.

Suggestions for Improvement:

- Add a comprehensive docstring explaining the function's purpose, arguments, and return value.
- Ensure consistent and correct indentation for all code blocks within the function.
- Use separate lines for `return` statements for better readability.
- Implement error handling for division by zero within the "div" condition.
- Rename the function and variables to be more descriptive, following PEP 8 naming conventions (e.g., `snake_case` for function and variable names).
- Add spaces around operators and after commas for improved readability, as per PEP 8.

EXPLANATION:

This cell provides a detailed review of the Python code in the `calc` function (which was in cell `CQgMj kpONyFd`). It points out several areas where the code could be improved for readability, maintainability, and robustness.

Here's a summary of what the report covers:

- **Issues Identified:** This section lists the problems found in the original code, such as missing documentation (docstrings), inconsistent code style (indentation, how `return` is used), lack of error handling for specific situations (like division by zero), and using names for functions and variables that don't clearly describe what they do.
- **Suggestions for Improvement:** This section offers concrete recommendations on how to fix the identified issues. This includes adding docstrings, ensuring consistent indentation and formatting according to standards like PEP 8, adding error handling, and using more descriptive names for better understanding.

Essentially, the report acts as a guide for making the `calc` function code cleaner, easier to understand, and less prone to errors.

ERROR CORRECTIONS:

V Please explain this error:

❗ File `"/tmp/ipython-input-4161972190.py"`, line 2 `if z=="add"...`

✦ The error in the selected cell (`CQgMjKpONyFd`) is an `IndentationError`: expected an indented block after function definition on line 1. This error occurs because the lines of code inside the `calc` function are not correctly indented. Python uses indentation to define code blocks, and all the code belonging to a function must be indented at the same level. I will fix the indentation to resolve this error.

It looks like fixing the indentation revealed another error: a `ZeroDivisionError: division by zero`. This happens when you try to divide a number by zero, which is mathematically undefined.